

Problem Statement

Business Context

Business communities in the United States are facing high demand for human resources, but one of the constant challenges is identifying and attracting the right talent, which is perhaps the most important element in remaining competitive. Companies in the United States look for hard-working, talented, and qualified individuals both locally as well as abroad.

The Immigration and Nationality Act (INA) of the US permits foreign workers to come to the United States to work on either a temporary or permanent basis. The act also protects US workers against adverse impacts on their wages or working conditions by ensuring US employers' compliance with statutory requirements when they hire foreign workers to fill workforce shortages. The immigration programs are administered by the Office of Foreign Labor Certification (OFLC).

OFLC processes job certification applications for employers seeking to bring foreign workers into the United States and grants certifications in those cases where employers can demonstrate that there are not sufficient US workers available to perform the work at wages that meet or exceed the wage paid for the occupation in the area of intended employment.

Objective

In FY 2016, the OFLC processed 775,979 employer applications for 1,699,957 positions for temporary and permanent labor certifications. This was a nine percent increase in the overall number of processed applications from the previous year. The process of reviewing every case is becoming a tedious task as the number of applicants is increasing every year.

The increasing number of applicants every year calls for a Machine Learning based solution that can help in shortlisting the candidates having higher chances of VISA approval. OFLC has hired the firm EasyVisa for data-driven solutions. You as a data scientist at EasyVisa have to analyze the data provided and, with the help of a classification model:

- Facilitate the process of visa approvals.
- Recommend a suitable profile for the applicants for whom the visa should be certified or denied based on the drivers that significantly influence the case status.

Data Description

The data contains the different attributes of employee and the employer. The detailed data dictionary is given below.

- **case_id:** ID of each visa application
- **continent:** Information of continent the employee
- **education_of_employee:** Information of education of the employee
- **has_job_experience:** Does the employee has any job experience? Y= Yes; N = No
- **requires_job_training:** Does the employee require any job training? Y = Yes; N = No
- **no_of_employees:** Number of employees in the employer's company
- **yr_of_estab:** Year in which the employer's company was established
- **region_of_employment:** Information of foreign worker's intended region of employment in the US.
- **prevailing_wage:** Average wage paid to similarly employed workers in a specific occupation in the area of intended employment. The purpose of the prevailing wage is to ensure that the foreign worker is not underpaid compared to other workers offering the same or similar service in the same area of employment.
- **unit_of_wage:** Unit of prevailing wage. Values include Hourly, Weekly, Monthly, and Yearly.
- **full_time_position:** Is the position of work full-time? Y = Full Time Position; N = Part Time Position
- **case_status:** Flag indicating if the Visa was certified or denied

Please read the instructions carefully before starting the project.

This is a commented Python Notebook file in which all the instructions and tasks to be performed are mentioned.

- Blanks "____" are provided in the notebook that needs to be filled with an appropriate code to get the correct result. With every "____" blank, there is a comment that briefly describes what needs to be filled in the blank space.
- Identify the task to be performed correctly, and only then proceed to write the required code.
- Please run the codes in a sequential manner from the beginning to avoid any unnecessary errors.
- Add the results/observations derived from the analysis in the presentation and submit the same.
 - Any mathematical or computational details which are a graded part of the project can be included in the *Appendix* section of the presentation.

Installing and Importing the necessary libraries

```
In [127... # Installing the Libraries with the specified version.
%pip install numpy==1.25.2 pandas==1.5.3 scikit-learn==1.5.2 matplotlib==3.7.1 seaborn
```

Note: you may need to restart the kernel to use updated packages.

```
[notice] A new release of pip is available: 25.2 -> 26.0.1
[notice] To update, run: c:\Users\JohnAraujo\.pyenv\pyenv-win\versions\3.11.0\python.exe -m pip install --upgrade pip
```

Note:

- After running the above cell, kindly restart the runtime (for Google Colab) or notebook kernel (for Jupyter Notebook), and run all cells sequentially from the next cell.
- On executing the above line of code, you might see a warning regarding package dependencies. This error message can be ignored as the above code ensures that all necessary libraries and their dependencies are maintained to successfully execute the code in **this notebook**.

```
In [128... # Libraries to help with reading and manipulating data
import numpy as np
import pandas as pd

# Library to split data
from sklearn.model_selection import train_test_split

# To oversample and undersample data
from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_sc

# Libraries to help with data visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Removes the limit for the number of displayed columns
pd.set_option("display.max_columns", None)
# Sets the limit for the number of displayed rows
pd.set_option("display.max_rows", 100)

# Libraries different ensemble classifiers
from sklearn.ensemble import (
    BaggingClassifier,
    RandomForestClassifier,
    AdaBoostClassifier,
    GradientBoostingClassifier
)

from xgboost import XGBClassifier
from sklearn.tree import DecisionTreeClassifier

# Libraries to get different metric scores
from sklearn import metrics
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
```

```

        f1_score,
    )

    # To tune different models
    from sklearn.model_selection import RandomizedSearchCV

    # To ignore unnecessary warnings
    import warnings
    warnings.filterwarnings("ignore")

```

Import Dataset

In [129... *# I am using VSCODE for it is convenient for me.*

Uncomment and execute the code snippets below if the data is stored in Google Drive
from google.colab import drive
drive.mount('/content/drive')

In [130... *# I am using VSCODE for it is convenient for me.*

```
visa = pd.read_csv('EasyVisa.csv') ## Complete the code to read the data
```

In [131... *# Copying data to another variable to avoid making changes to the original data*

```
data = visa.copy()
```

Overview of the Dataset

View the first and last 5 rows of the dataset

In [132... `data.head()` *## Complete the code to view top 5 rows of the data*

Out[132...

	case_id	continent	education_of_employee	has_job_experience	requires_job_training	no
0	EZYV01	Asia	High School	N	N	
1	EZYV02	Asia	Master's	Y	N	
2	EZYV03	Asia	Bachelor's	N	Y	
3	EZYV04	Asia	Bachelor's	N	N	
4	EZYV05	Africa	Master's	Y	N	



In [133... `data.tail()` *## Complete the code to view last 5 rows of the data*

Out[133...

	case_id	continent	education_of_employee	has_job_experience	requires_job_train
25475	EZYV25476	Asia	Bachelor's	Y	
25476	EZYV25477	Asia	High School	Y	
25477	EZYV25478	Asia	Master's	Y	
25478	EZYV25479	Asia	Master's	Y	
25479	EZYV25480	Asia	Bachelor's	Y	



Understand the shape of the dataset

In [134...

```
data.shape ## Complete the code to view dimensions of the data
```

Out[134...

(25480, 12)

- The dataset has 25480 rows and 12 columns

Check the data types of the columns for the dataset

In [135...

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25480 entries, 0 to 25479
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   case_id               25480 non-null  object
 1   continent             25480 non-null  object
 2   education_of_employee 25480 non-null  object
 3   has_job_experience     25480 non-null  object
 4   requires_job_training 25480 non-null  object
 5   no_of_employees       25480 non-null  int64
 6   yr_of_estab           25480 non-null  int64
 7   region_of_employment 25480 non-null  object
 8   prevailing_wage       25480 non-null  float64
 9   unit_of_wage          25480 non-null  object
10   full_time_position    25480 non-null  object
11   case_status           25480 non-null  object
dtypes: float64(1), int64(2), object(9)
memory usage: 2.3+ MB
```

In [136...

```
# checking for duplicate values
data.duplicated().sum() ## Complete the code to check duplicate entries in the dat
```

Out[136...

0

Exploratory Data Analysis (EDA)

Let's check the statistical summary of the data

In [137... `data.describe().T` ## Complete the code to print the statistical summary of the dat

Out[137...

	count	mean	std	min	25%	50%	
no_of_employees	25480.0	5667.043210	22877.928848	-26.0000	1022.00	2109.00	35
yr_of_estab	25480.0	1979.409929	42.366929	1800.0000	1976.00	1997.00	20
prevailing_wage	25480.0	74455.814592	52815.942327	2.1367	34015.48	70308.21	1077



Fixing the negative values in number of employees columns

In [138... `data.loc[data["no_of_employees"] < 0].shape` ## Complete the code to check negative

Out[138... (33, 12)

In [139... `# taking the absolute values for number of employees`
`data["no_of_employees"] = abs(data["no_of_employees"])`

Let's check the count of each unique category in each of the categorical variables

In [140... `# Making a list of all catrgorical variables`
`cat_col = list(data.select_dtypes("object").columns)`

`# Printing number of count of each unique value in each column`
`for column in cat_col:`
 `print(data[column].value_counts())`
 `print("-" * 50)`

```

EZYV01      1
EZYV16995   1
EZYV16993   1
EZYV16992   1
EZYV16991   1
..
EZYV8492    1
EZYV8491    1
EZYV8490    1
EZYV8489    1
EZYV25480   1
Name: case_id, Length: 25480, dtype: int64

```

```

-----
Asia          16861
Europe        3732
North America 3292
South America 852
Africa         551
Oceania        192
Name: continent, dtype: int64

```

```

-----
Bachelor's    10234
Master's      9634
High School   3420
Doctorate     2192
Name: education_of_employee, dtype: int64

```

```

-----
Y    14802
N    10678
Name: has_job_experience, dtype: int64

```

```

-----
N    22525
Y    2955
Name: requires_job_training, dtype: int64

```

```

-----
Northeast     7195
South          7017
West           6586
Midwest        4307
Island         375
Name: region_of_employment, dtype: int64

```

```

-----
Year          22962
Hour           2157
Week           272
Month           89
Name: unit_of_wage, dtype: int64

```

```

-----
Y    22773
N    2707
Name: full_time_position, dtype: int64

```

```

-----
Certified     17018
Denied         8462
Name: case_status, dtype: int64
-----

```

Education to case_status ratio

```
In [141... # print table of percentage of education and approval denial
result = (
    data.groupby("has_job_experience")["case_status"]
    .value_counts(normalize=True)
    .unstack(fill_value=0)
    .mul(100).round(2)
)
#result["Certified %"] = result["Certified"]
#result["Denied %"] = result["Denied"]
result = result.rename(columns={"Certified": "Certified %", "Denied": "Denied %"})
result.columns.name = None
print (result)
```

	Certified %	Denied %
has_job_experience		
N	56.13	43.87
Y	74.48	25.52

```
In [142... # print table of percentage of education and approval denial
result = (
    data.groupby("education_of_employee")["case_status"]
    .value_counts(normalize=True)
    .unstack(fill_value=0)
    .mul(100).round(2)
)
#result["Certified %"] = result["Certified"]
#result["Denied %"] = result["Denied"]
result = result.rename(columns={"Certified": "Certified %", "Denied": "Denied %"})
result.columns.name = None
print (result)
```

	Certified %	Denied %
education_of_employee		
Bachelor's	62.21	37.79
Doctorate	87.23	12.77
High School	34.04	65.96
Master's	78.63	21.37

```
In [143... # checking the number of unique values
data["case_id"].unique ## Complete the code to check unique values in the mentioned
```

```
Out[143... <bound method Series.unique of 0          EZYV01
1          EZYV02
2          EZYV03
3          EZYV04
4          EZYV05
...
25475      EZYV25476
25476      EZYV25477
25477      EZYV25478
25478      EZYV25479
25479      EZYV25480
Name: case_id, Length: 25480, dtype: object>
```



```
In [144... data.drop(["case_id"], axis=1, inplace=True) ## Complete the code to drop 'case_id'
```

Univariate Analysis

```
In [145... def histogram_boxplot(data, feature, figsize=(15, 10), kde=False, bins=None):
    """
    Boxplot and histogram combined

    data: dataframe
    feature: dataframe column
    figsize: size of figure (default (15,10))
    kde: whether to show the density curve (default False)
    bins: number of bins for histogram (default None)
    """

    f2, (ax_box2, ax_hist2) = plt.subplots(
        nrows=2, # Number of rows of the subplot grid= 2
        sharex=True, # x-axis will be shared among all subplots
        gridspec_kw={"height_ratios": (0.25, 0.75)},
        figsize=figsize,
    ) # creating the 2 subplots
    sns.boxplot(
        data=data, x=feature, ax=ax_box2, showmeans=True, color="violet"
    ) # boxplot will be created and a triangle will indicate the mean value of the
    sns.histplot(
        data=data, x=feature, kde=kde, ax=ax_hist2, bins=bins
    ) if bins else sns.histplot(
        data=data, x=feature, kde=kde, ax=ax_hist2
    ) # For histogram
    ax_hist2.axvline(
        data[feature].mean(), color="green", linestyle="--"
    ) # Add mean to the histogram
    ax_hist2.axvline(
        data[feature].median(), color="black", linestyle="-"
    ) # Add median to the histogram
```

```
In [146... # function to create labeled barplots

def labeled_barplot(data, feature, perc=False, n=None):
    """
    Barplot with percentage at the top

    data: dataframe
    feature: dataframe column
    perc: whether to display percentages instead of count (default is False)
    n: displays the top n category levels (default is None, i.e., display all level
    """

    total = len(data[feature]) # Length of the column
    count = data[feature].nunique()
    if n is None:
        plt.figure(figsize=(count + 1, 5))
    else:
        plt.figure(figsize=(n + 1, 5))
```

```

plt.xticks(rotation=90, fontsize=15)
ax = sns.countplot(
    data=data,
    x=feature,
    palette="Paired",
    order=data[feature].value_counts().index[:n].sort_values(),
)

for p in ax.patches:
    if perc == True:
        label = "{:.1f}%".format(
            100 * p.get_height() / total
        ) # percentage of each class of the category
    else:
        label = p.get_height() # count of each level of the category

    x = p.get_x() + p.get_width() / 2 # width of the plot
    y = p.get_height() # height of the plot

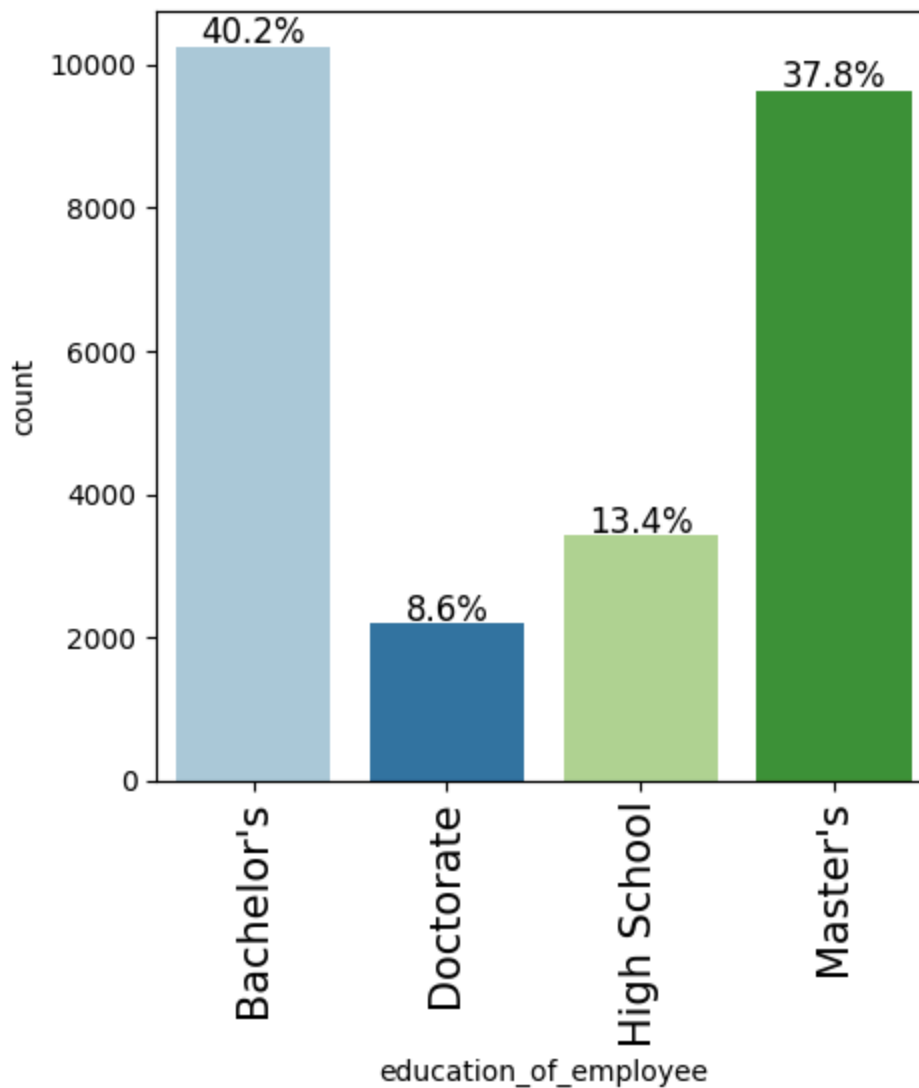
    ax.annotate(
        label,
        (x, y),
        ha="center",
        va="center",
        size=12,
        xytext=(0, 5),
        textcoords="offset points",
    ) # annotate the percentage

plt.show() # show the plot

```

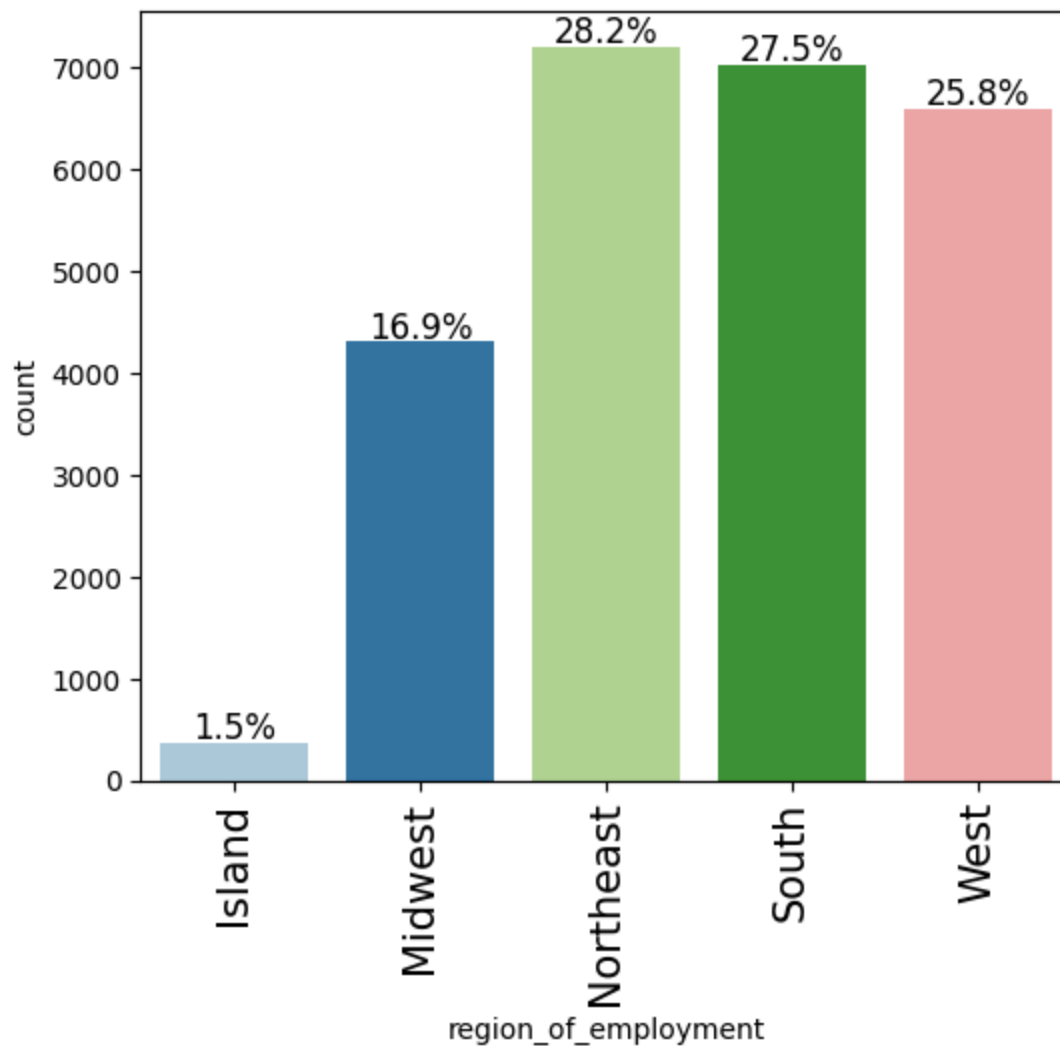
Observations on education of employee

In [147... `labeled_barplot(data, "education_of_employee", perc=True)`



Observations on region of employment

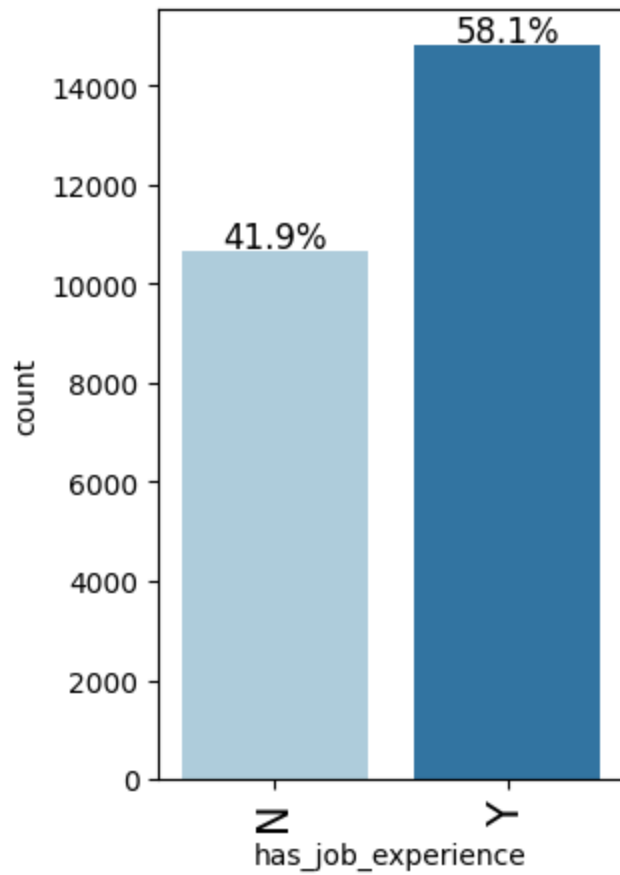
In [148... `labeled_barplot(data, "region_of_employment", perc=True)` *## Complete the code to cr*



Observations on job experience

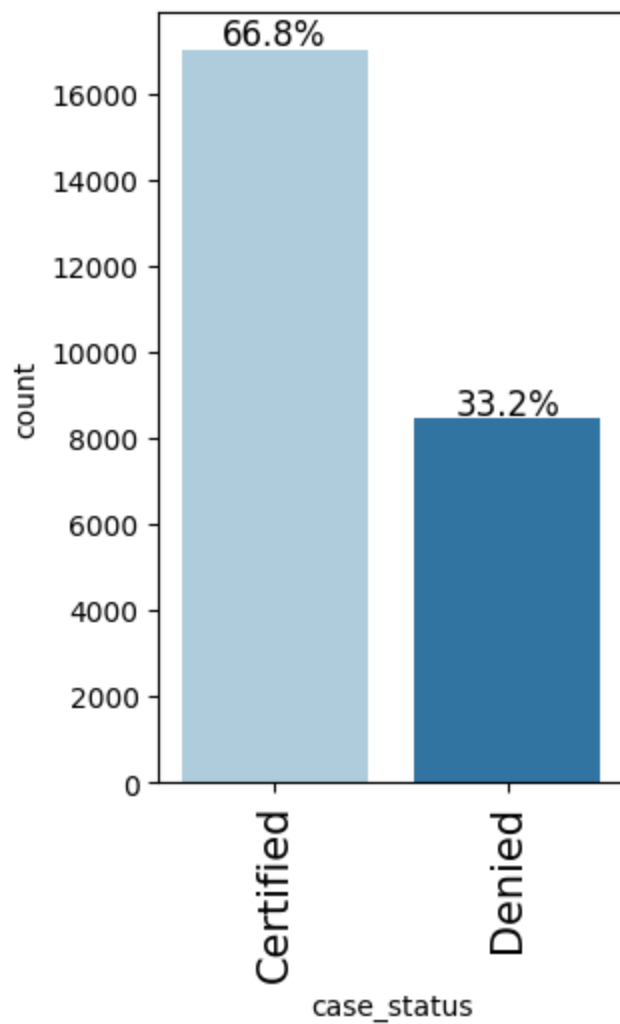
In [149...

```
labeled_barplot(data, "has_job_experience", perc=True) ## Complete the code to create
```



Observations on case status

In [150... `labeled_barplot(data, "case_status", perc=True) ## Complete the code to create labeled_barplot`

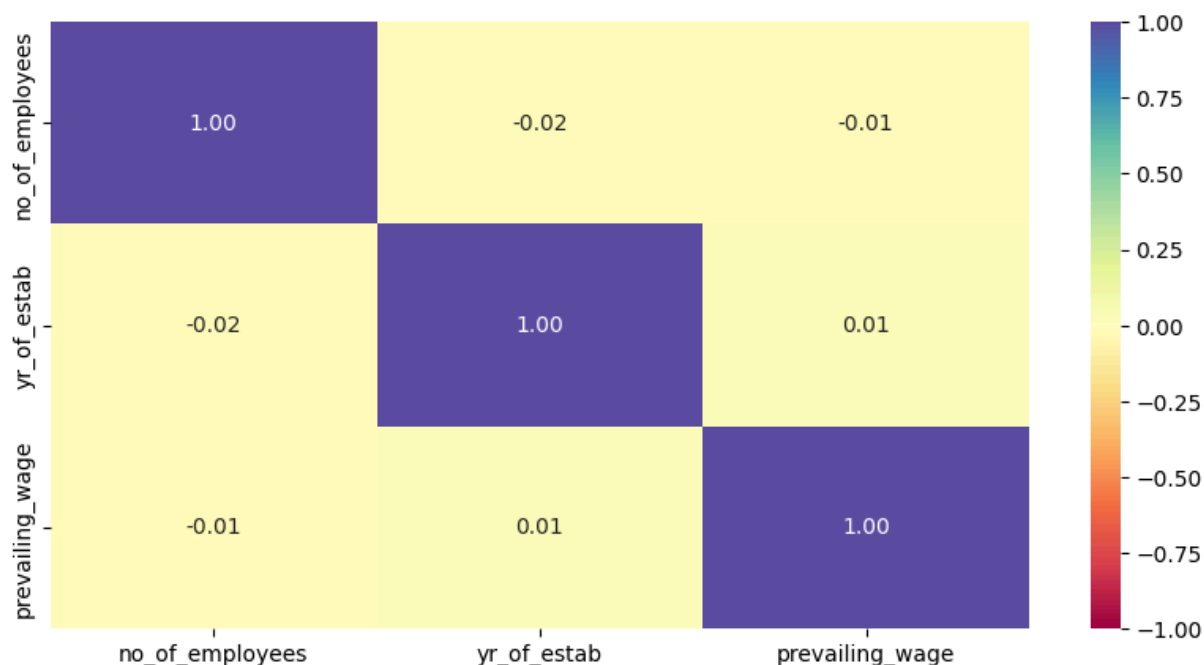


Bivariate Analysis

Correlation Check

```
In [151... cols_list = data.select_dtypes(include=np.number).columns.tolist()

plt.figure(figsize=(10, 5))
sns.heatmap(
    data[cols_list].corr(), annot=True, vmin=-1, vmax=1, fmt=".2f", cmap="Spectral"
)
plt.show()
```



Creating functions that will help us with further analysis.

```
In [152... ### function to plot distributions wrt target

def distribution_plot_wrt_target(data, predictor, target):

    fig, axs = plt.subplots(2, 2, figsize=(12, 10))

    target_uniq = data[target].unique()

    axs[0, 0].set_title("Distribution of target for target=" + str(target_uniq[0]))
    sns.histplot(
        data=data[data[target] == target_uniq[0]],
        x=predictor,
        kde=True,
        ax=axs[0, 0],
        color="teal",
        stat="density",
    )

    axs[0, 1].set_title("Distribution of target for target=" + str(target_uniq[1]))
    sns.histplot(
        data=data[data[target] == target_uniq[1]],
        x=predictor,
        kde=True,
        ax=axs[0, 1],
        color="orange",
        stat="density",
    )

    axs[1, 0].set_title("Boxplot w.r.t target")
    sns.boxplot(data=data, x=target, y=predictor, ax=axs[1, 0], palette="gist_rainb
```

```

axs[1, 1].set_title("Boxplot (without outliers) w.r.t target")
sns.boxplot(
    data=data,
    x=target,
    y=predictor,
    ax=axs[1, 1],
    showfliers=False,
    palette="gist_rainbow",
)

plt.tight_layout()
plt.show()

```

```

In [153... def stacked_barplot(data, predictor, target):
    """
    Print the category counts and plot a stacked bar chart

    data: dataframe
    predictor: independent variable
    target: target variable
    """
    count = data[predictor].nunique()
    sorter = data[target].value_counts().index[-1]
    tab1 = pd.crosstab(data[predictor], data[target], margins=True).sort_values(
        by=sorter, ascending=False
    )
    print(tab1)
    print("-" * 120)
    tab = pd.crosstab(data[predictor], data[target], normalize="index").sort_values(
        by=sorter, ascending=False
    )
    tab.plot(kind="bar", stacked=True, figsize=(count + 5, 5))
    plt.legend(
        loc="lower left", frameon=False,
    )
    plt.legend(loc="upper left", bbox_to_anchor=(1, 1))
    plt.show()

```

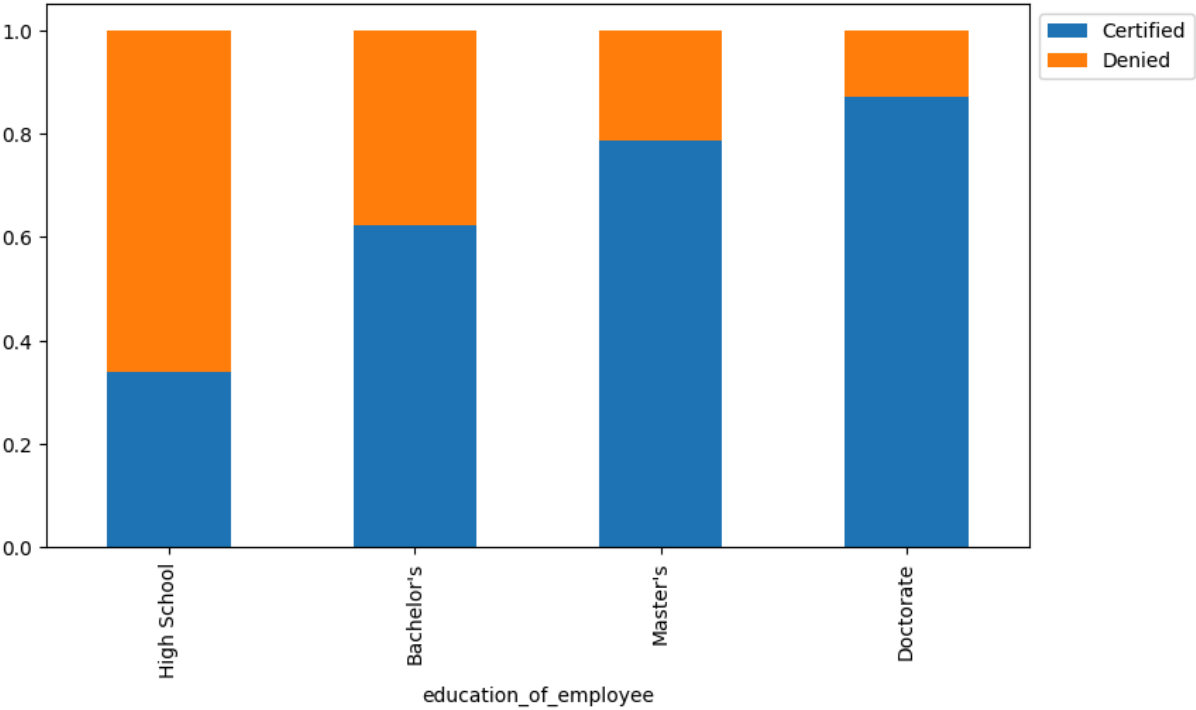
Does higher education increase the chances of visa certification for well-paid jobs abroad?

```

In [154... stacked_barplot(data, "education_of_employee", "case_status")

```

case_status	Certified	Denied	All
education_of_employee			
All	17018	8462	25480
Bachelor's	6367	3867	10234
High School	1164	2256	3420
Master's	7575	2059	9634
Doctorate	1912	280	2192

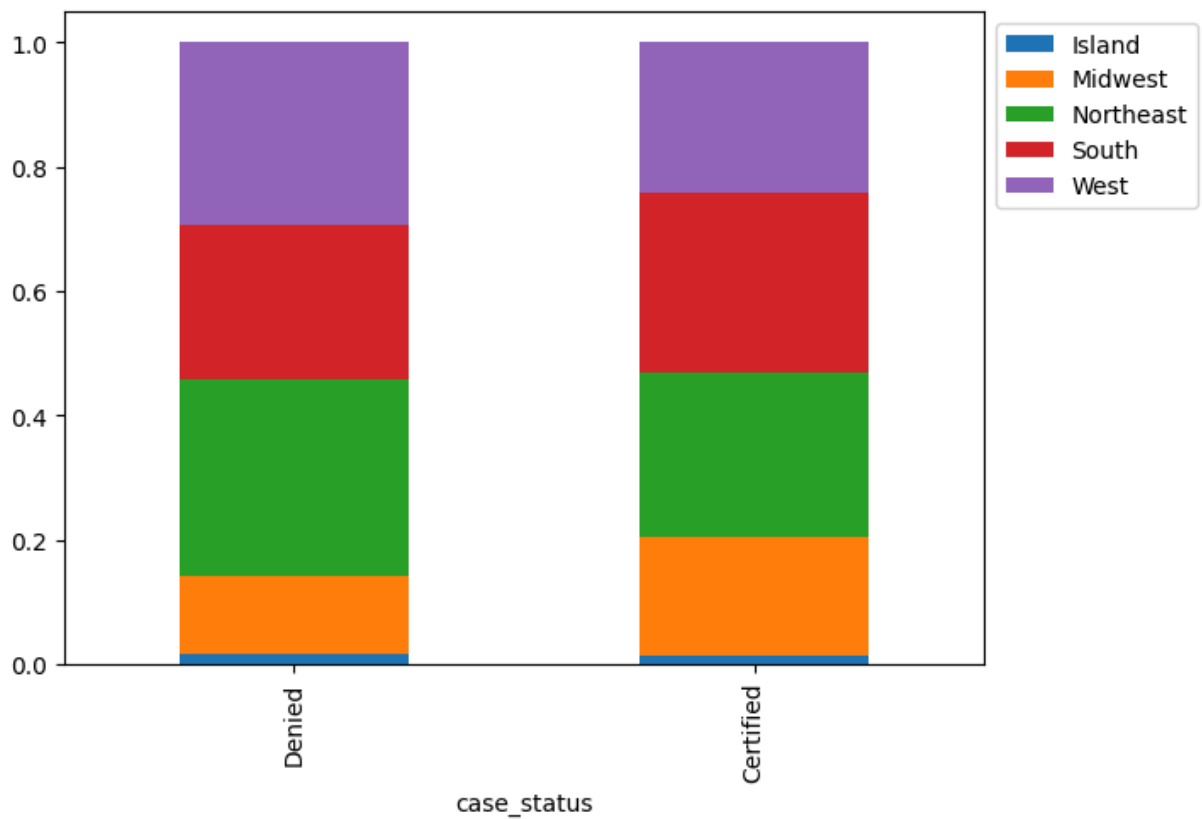


How does visa status vary across different continents?

In [155...

```
stacked_barplot(data,"case_status","region_of_employment") ## Complete the code to
```

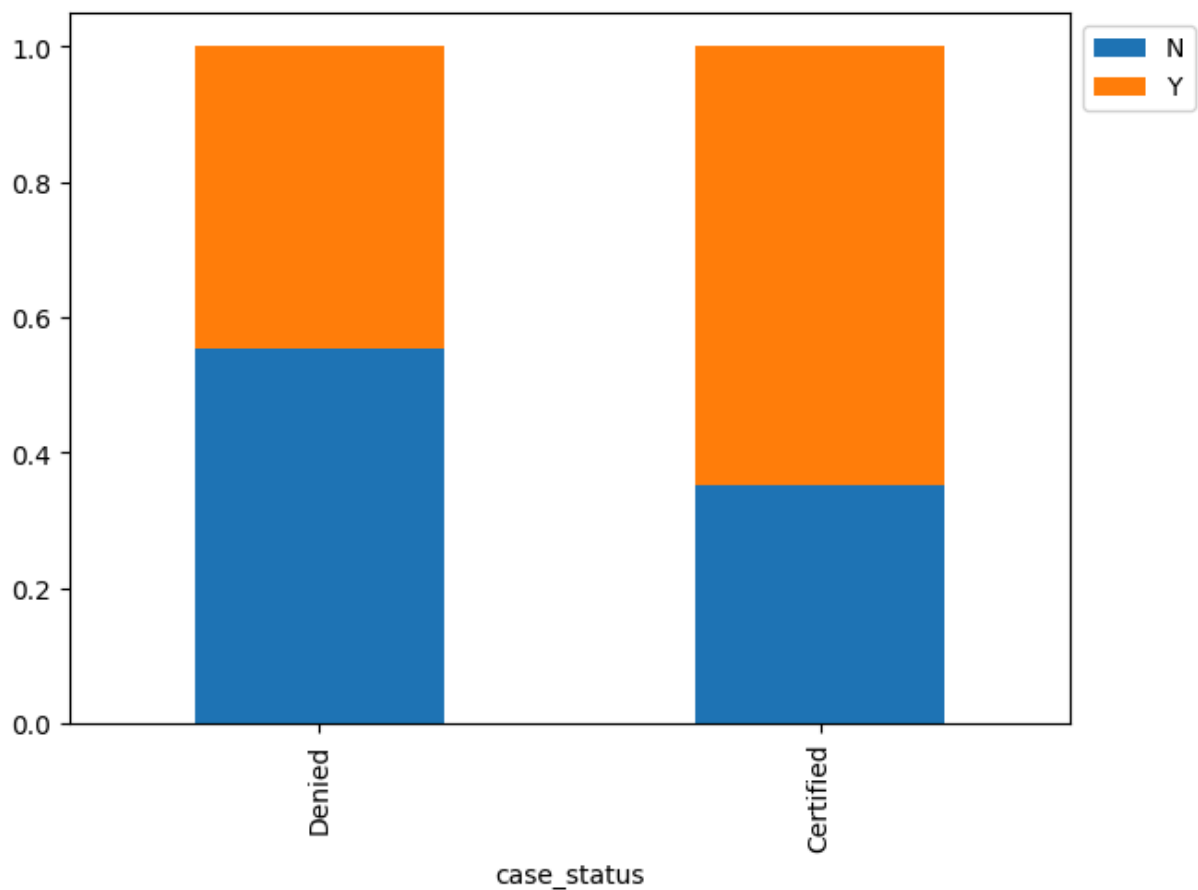
region_of_employment	Island	Midwest	Northeast	South	West	All
case_status						
All	375	4307	7195	7017	6586	25480
Certified	226	3253	4526	4913	4100	17018
Denied	149	1054	2669	2104	2486	8462



Does having prior work experience influence the chances of visa certification for career opportunities abroad?

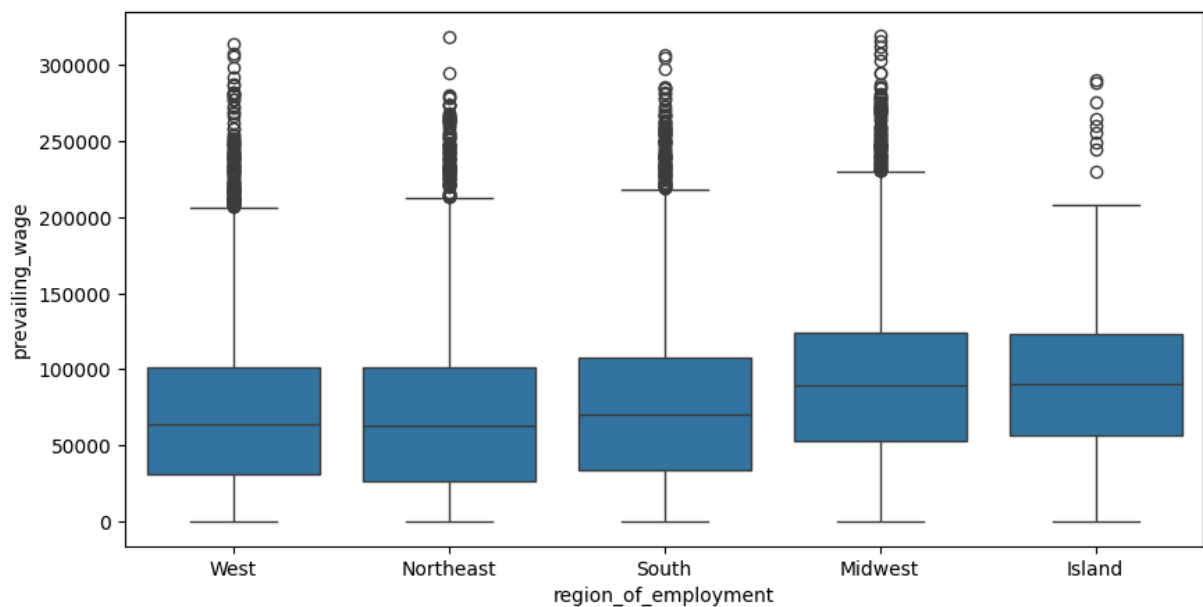
In [156... `stacked_barplot(data, "case_status", "has_job_experience")` ## Complete the code to pl

has_job_experience	N	Y	All
case_status			
All	10678	14802	25480
Certified	5994	11024	17018
Denied	4684	3778	8462



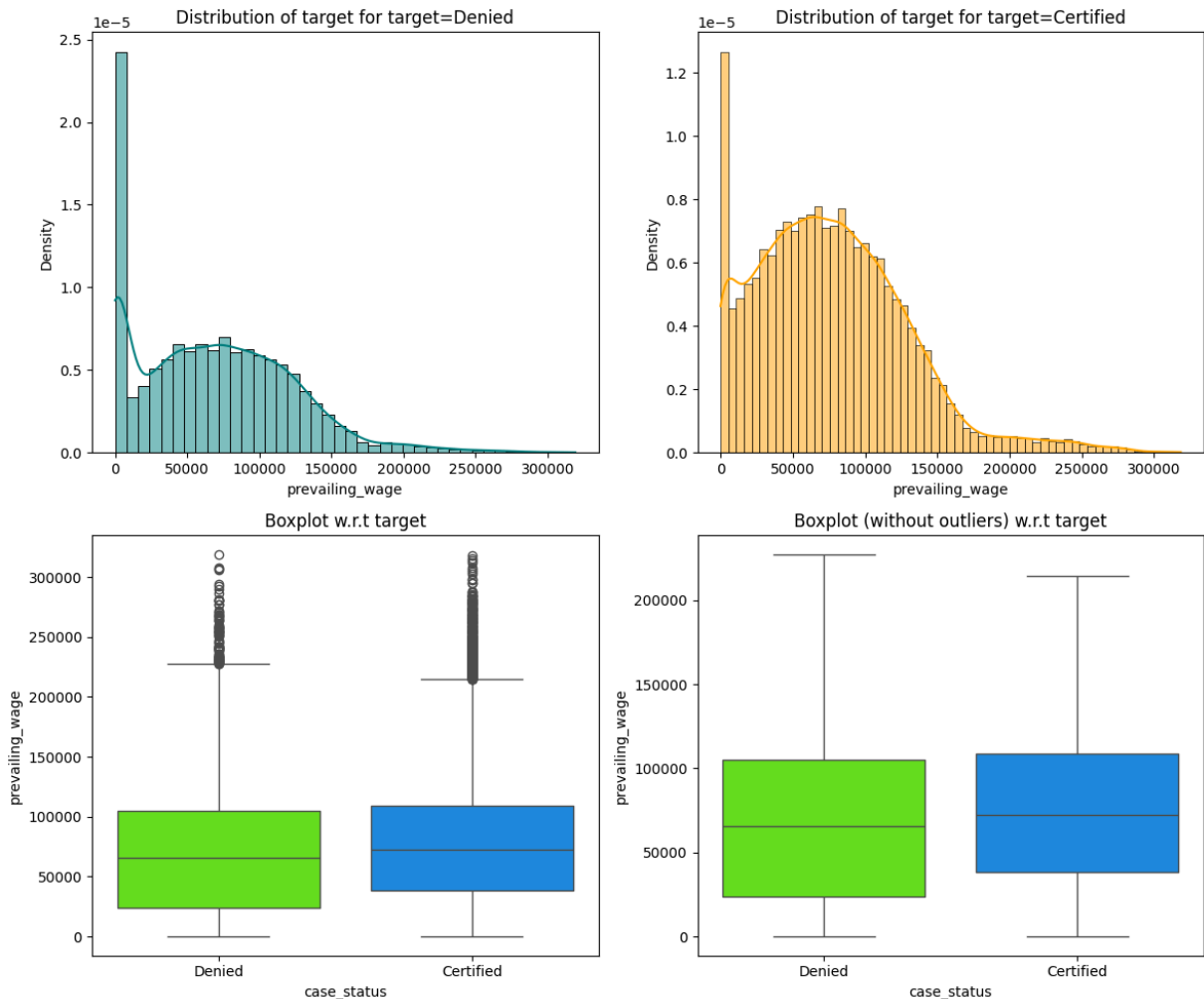
Is the prevailing wage consistent across all regions of the US?

```
In [157... plt.figure(figsize=(10, 5))
sns.boxplot(data=data, x="region_of_employment", y="prevailing_wage") ## Complete t
plt.show()
```



Does visa status vary with changes in the prevailing wage set to protect both local talent and foreign workers?

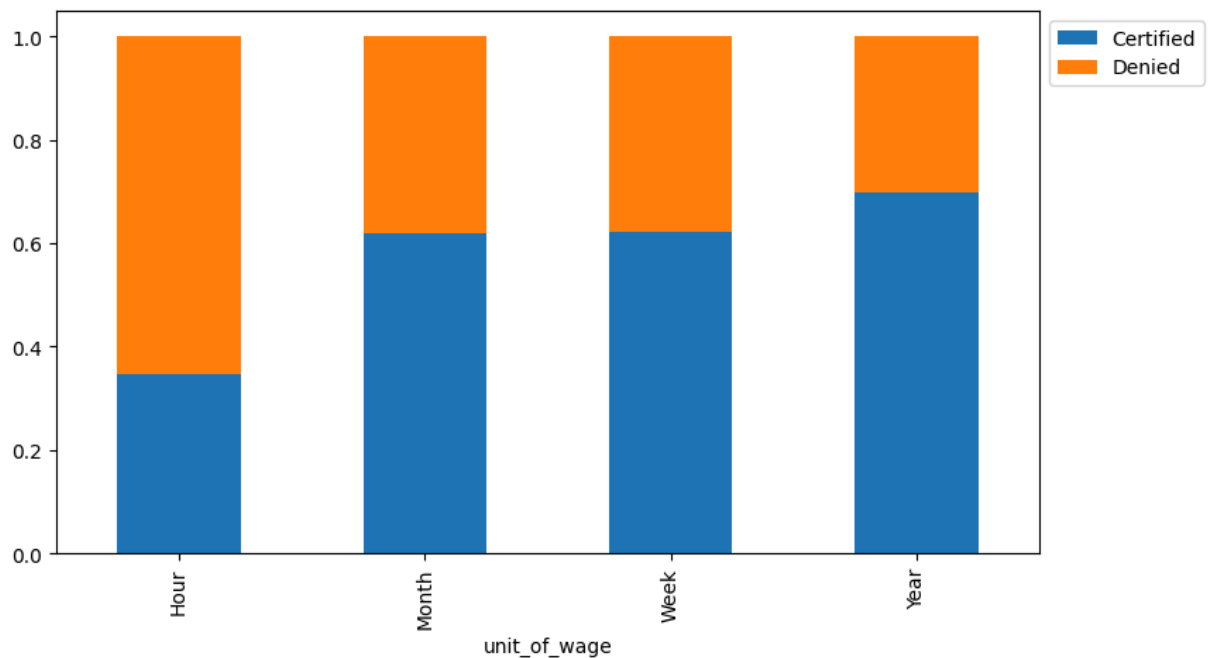
In [158... `distribution_plot_wrt_target(data,"prevailing_wage","case_status")` *## Complete the*



Does the unit of prevailing wage (Hourly, Weekly, etc.) have any impact on the likelihood of visa application certification?

In [159... `stacked_barplot(data,"unit_of_wage","case_status")` *## Complete the code to plot sta*

case_status	Certified	Denied	All
unit_of_wage			
All	17018	8462	25480
Year	16047	6915	22962
Hour	747	1410	2157
Week	169	103	272
Month	55	34	89



Data Pre-processing

Outlier Check

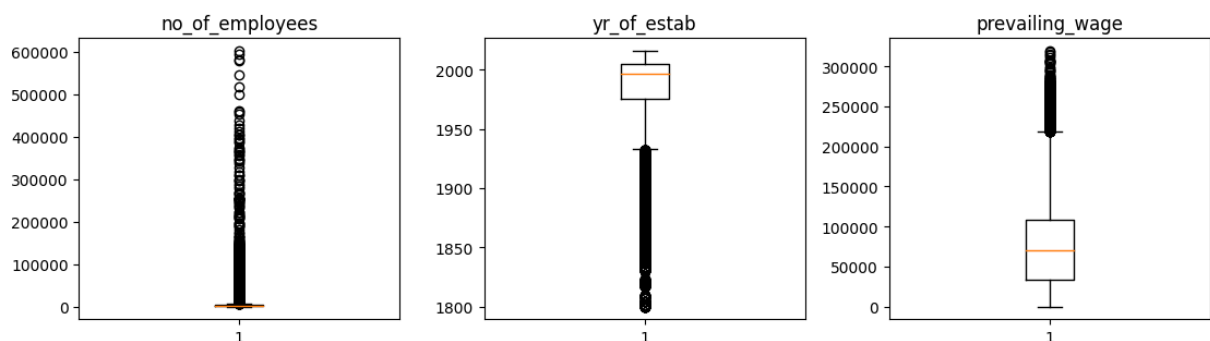
- Let's check for outliers in the data.

```
In [160... # outlier detection using boxplot
numeric_columns = data.select_dtypes(include=np.number).columns.tolist()

plt.figure(figsize=(15, 12))

for i, variable in enumerate(numeric_columns):
    plt.subplot(4, 4, i + 1)
    plt.boxplot(data[variable], whis=1.5)
    plt.tight_layout()
    plt.title(variable)

plt.show()
```



Data Preparation for modeling

- We want to predict which visa will be certified.
- Before we proceed to build a model, we'll have to encode categorical features.
- We'll split the data into train and test to be able to evaluate the model that we build on the train data.

```
In [161... data["case_status"] = data["case_status"].apply(lambda x: 1 if x == "Certified" else 0)

X = data.drop(['case_status'],axis=1) ## Complete the code to drop case status from data
y = data["case_status"]

X = pd.get_dummies(X,drop_first=True)

# Complete the code to split the dataset into train and valid with a ratio of 7:3
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.30, random_state=1, stratify=y
)

## Complete the code to split the dataset into valid and test with a ratio of 9:1
X_val,X_test,y_val,y_test = train_test_split(
    X_val,y_val,test_size=0.10,random_state=1,stratify=y_val
)
```

```
In [162... print("Shape of Training set : ", X_train.shape)
print("Shape of Validation set : ", X_val.shape)
print("Shape of test set : ", X_test.shape)
print("Percentage of classes in training set:")
print(y_train.value_counts(normalize=True))
print("Percentage of classes in validation set:")
print(y_val.value_counts(normalize=True))
print("Percentage of classes in test set:")
print(y_test.value_counts(normalize=True))
```

```
Shape of Training set : (17836, 21)
Shape of Validation set : (6879, 21)
Shape of test set : (765, 21)
Percentage of classes in training set:
1    0.667919
0    0.332081
Name: case_status, dtype: float64
Percentage of classes in validation set:
1    0.66783
0    0.33217
Name: case_status, dtype: float64
Percentage of classes in test set:
1    0.667974
0    0.332026
Name: case_status, dtype: float64
```

Model Building

Model Evaluation Criterion

- Choose the primary metric to evaluate the model on
- Elaborate on the rationale behind choosing the metric

First, let's create functions to calculate different metrics and confusion matrix so that we don't have to use the same code repeatedly for each model.

- The `model_performance_classification_sklearn` function will be used to check the model performance of models.
- The `confusion_matrix_sklearn` function will be used to plot the confusion matrix.

```
In [163... # defining a function to compute different metrics to check performance of a classi

def model_performance_classification_sklearn(model, predictors, target):
    """
    Function to compute different metrics to check classification model performance

    model: classifier
    predictors: independent variables
    target: dependent variable
    """

    # predicting using the independent variables
    pred = model.predict(predictors)

    acc = accuracy_score(target, pred) # to compute Accuracy
    recall = recall_score(target, pred) # to compute Recall
    precision = precision_score(target, pred) # to compute Precision
    f1 = f1_score(target, pred) # to compute F1-score

    # creating a dataframe of metrics
    df_perf = pd.DataFrame(
        {"Accuracy": acc, "Recall": recall, "Precision": precision, "F1": f1},
        index=[0],
    )

    return df_perf
```

```
In [164... def confusion_matrix_sklearn(model, predictors, target):
    """
    To plot the confusion_matrix with percentages

    model: classifier
    predictors: independent variables
    target: dependent variable
    """

    y_pred = model.predict(predictors)
    cm = confusion_matrix(target, y_pred)
    labels = np.asarray(
        [
            "{0:0.0f}".format(item) + "\n{0:.2%}".format(item / cm.flatten().sum())
            for item in cm.flatten()
        ]
    )
```

```

    ]
).reshape(2, 2)

plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=labels, fmt="")
plt.ylabel("True label")
plt.xlabel("Predicted label")

```

Defining scorer to be used for cross-validation and hyperparameter tuning

```

In [169... scorer = metrics.make_scorer(metrics.f1_score) ## Complete the code to define the m

## Possible metrics are [recall_score, f1_score, accuracy_score, precision_score]
## For example, metrics.precision_score

```

We are now done with pre-processing and evaluation criterion, so let's start building the model.

Model building with Original data

```

In [174... models = [] # Empty list to store all the models

# Appending models into the list
models.append(("Bagging", BaggingClassifier(random_state=1)))
models.append(("Random forest", RandomForestClassifier(random_state=1)))
models.append(("GBM", GradientBoostingClassifier(random_state=1)))
models.append(("Adaboost", AdaBoostClassifier(random_state=1)))
models.append(("Xgboost", XGBClassifier(random_state=1, eval_metric="logloss")))
models.append(("dtree", DecisionTreeClassifier(random_state=1)))

results1 = [] # Empty list to store all model's CV scores
names = [] # Empty list to store name of the models

# Loop through all models to get the mean cross validated score
print("\n" "Cross-Validation performance on training dataset:" "\n")

for name, model in models:
    kfold = StratifiedKFold(
        n_splits=5, shuffle=True, random_state=1
    ) # Complete the code to set the number of splits.
    cv_result = cross_val_score(
        estimator=model, X=X_train, y=y_train, scoring = scorer, cv=kfold
    )
    results1.append(cv_result)
    names.append(name)
    print("{}: {}".format(name, cv_result.mean()))

print("\n" "Validation Performance:" "\n")

for name, model in models:
    model.fit(X_train, y_train) ## Complete the code to fit the model on original t

```



```
# scores = accuracy_score(y_val, model.predict(X_val)) ## uncomment this line i
# scores = recall_score(y_val, model.predict(X_val)) ## uncomment this line in
# scores = precision_score(y_val, model.predict(X_val)) ## uncomment this line
scores = f1_score(y_val, model.predict(X_val)) ## uncomment this line in case t

print("{}: {}".format(name, scores))
```

Cross-Validation performance on training dataset:

Bagging: 0.7756586246579394
 Random forest: 0.8037837241749051
 GBM: 0.823039791269532
 Adaboost: 0.8177228312991336
 Xgboost: 0.8073583989766158
 dtree: 0.7410652876513099

Validation Performance:

Bagging: 0.7675817565350541
 Random forest: 0.7972364702187794
 GBM: 0.8195818459969403
 Adaboost: 0.8151868220168742
 Xgboost: 0.8070320579110651
 dtree: 0.7477497255762898

In [175... *# Plotting boxplots for CV scores of all models defined above*

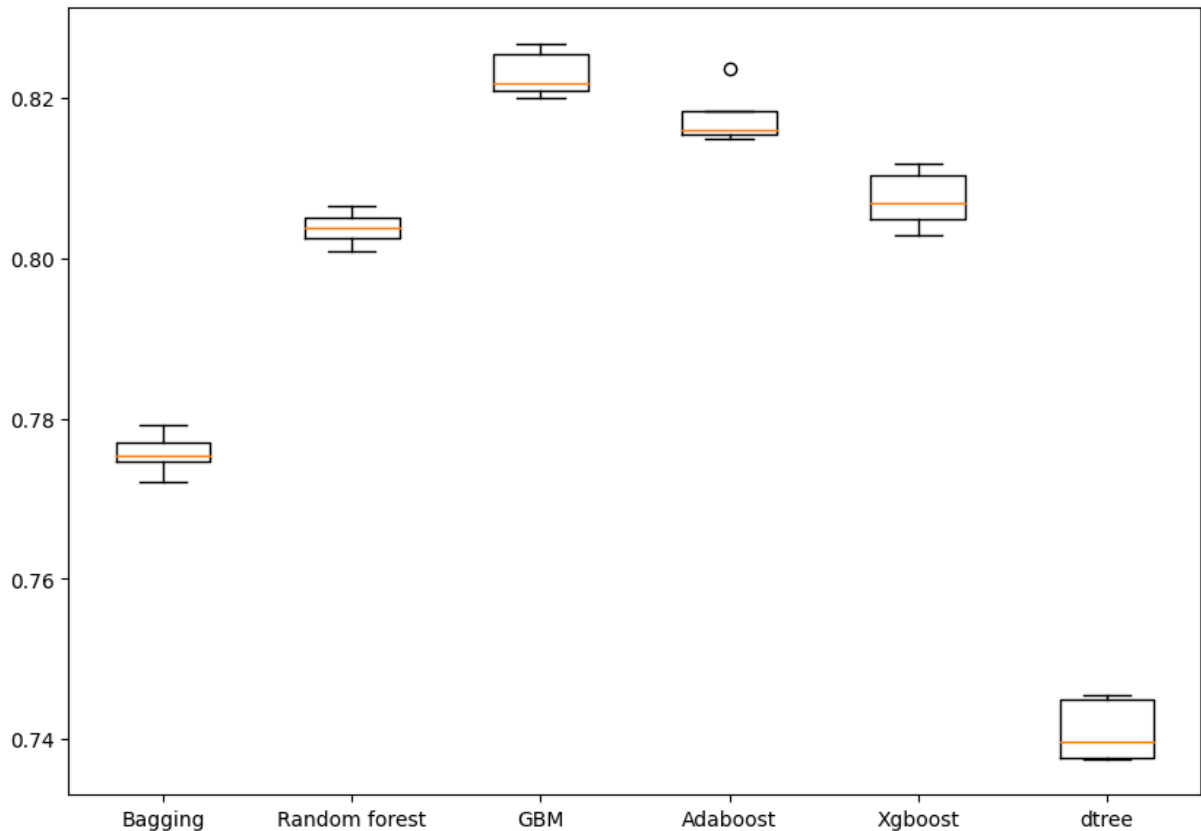
```
fig = plt.figure(figsize=(10, 7))

fig.suptitle("Algorithm Comparison")
ax = fig.add_subplot(111)

plt.boxplot(results1)
ax.set_xticklabels(names)

plt.show()
```

Algorithm Comparison



Model Building with Oversampled data

```
In [177... print("Before OverSampling, counts of label '1': {}".format(sum(y_train == 1)))
print("Before OverSampling, counts of label '0': {} \n".format(sum(y_train == 0)))

# Synthetic Minority Over Sampling Technique
sm = SMOTE(sampling_strategy=1, k_neighbors=5, random_state=1) ## Complete the code
X_train_over, y_train_over = sm.fit_resample(X_train, y_train)

print("After OverSampling, counts of label '1': {}".format(sum(y_train_over == 1)))
print("After OverSampling, counts of label '0': {} \n".format(sum(y_train_over == 0)))

print("After OverSampling, the shape of train_X: {}".format(X_train_over.shape))
print("After OverSampling, the shape of train_y: {} \n".format(y_train_over.shape))
```

Before OverSampling, counts of label '1': 11913

Before OverSampling, counts of label '0': 5923

After OverSampling, counts of label '1': 11913

After OverSampling, counts of label '0': 11913

After OverSampling, the shape of train_X: (23826, 21)

After OverSampling, the shape of train_y: (23826,)

In [180...

```

models = [] # Empty list to store all the models

# Appending models into the list
models.append(("Bagging", BaggingClassifier(random_state=1)))
models.append(("Random forest", RandomForestClassifier(random_state=1)))
models.append(("GBM", GradientBoostingClassifier(random_state=1)))
models.append(("Adaboost", AdaBoostClassifier(random_state=1)))
models.append(("Xgboost", XGBClassifier(random_state=1, eval_metric="logloss")))
models.append(("dtree", DecisionTreeClassifier(random_state=1)))

results1 = [] # Empty list to store all model's CV scores
names = [] # Empty list to store name of the models

# Loop through all models to get the mean cross validated score
print("\n" "Cross-Validation performance on training dataset:" "\n")

for name, model in models:
    kfold = StratifiedKFold(
        n_splits=5, shuffle=True, random_state=1
    ) ## Complete the code to set the number of splits
    cv_result = cross_val_score(
        estimator=model, X=X_train_over, y=y_train_over, scoring = scorer, cv=kfold
    )
    results1.append(cv_result)
    names.append(name)
    print("{}: {}".format(name, cv_result.mean()))

print("\n" "Validation Performance:" "\n")

for name, model in models:
    model.fit(X_train_over, y_train_over) ## Complete the code to fit the model on

    # scores = accuracy_score(y_val, model.predict(X_val)) ## uncomment this line i
    # scores = recall_score(y_val, model.predict(X_val)) ## uncomment this line in
    # scores = precision_score(y_val, model.predict(X_val)) ## uncomment this line
    scores = f1_score(y_val, model.predict(X_val)) ## uncomment this line in case t

    print("{}: {}".format(name, scores))

```

Cross-Validation performance on training dataset:

Bagging: 0.7630248470329939
Random forest: 0.789527508885066
GBM: 0.7995200290888033
Adaboost: 0.7853502765579534
Xgboost: 0.7947610182951765
dtree: 0.7302748433737385

Validation Performance:

Bagging: 0.7615769017212659
Random forest: 0.7873070325900514
GBM: 0.8013683985460766
Adaboost: 0.7846414216204781
Xgboost: 0.7933446375582874
dtree: 0.7402482269503546

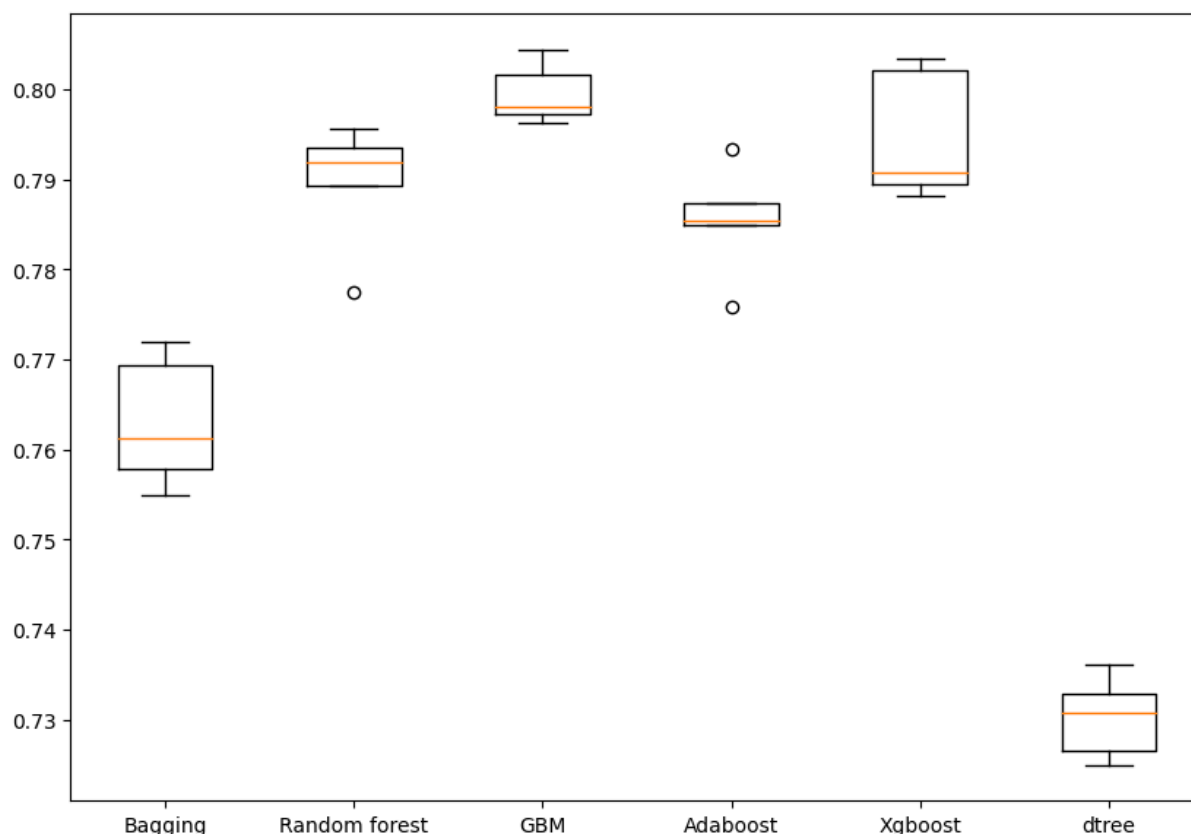
```
In [181... # Plotting boxplots for CV scores of all models defined above
fig = plt.figure(figsize=(10, 7))

fig.suptitle("Algorithm Comparison")
ax = fig.add_subplot(111)

plt.boxplot(results1)
ax.set_xticklabels(names)

plt.show()
```

Algorithm Comparison



Model Building with Undersampled data

```
In [182... rus = RandomUnderSampler(random_state=1, sampling_strategy=1)
X_train_un, y_train_un = rus.fit_resample(X_train, y_train)

print("Before UnderSampling, counts of label '1': {}".format(sum(y_train == 1)))
print("Before UnderSampling, counts of label '0': {} \n".format(sum(y_train == 0)))

print("After UnderSampling, counts of label '1': {}".format(sum(y_train_un == 1)))
print("After UnderSampling, counts of label '0': {} \n".format(sum(y_train_un == 0)))

print("After UnderSampling, the shape of train_X: {}".format(X_train_un.shape))
print("After UnderSampling, the shape of train_y: {} \n".format(y_train_un.shape))
```

Before UnderSampling, counts of label '1': 11913

Before UnderSampling, counts of label '0': 5923

After UnderSampling, counts of label '1': 5923

After UnderSampling, counts of label '0': 5923

After UnderSampling, the shape of train_X: (11846, 21)

After UnderSampling, the shape of train_y: (11846,)

In [186...

```

models = [] # Empty list to store all the models

# Appending models into the list
models.append(("Bagging", BaggingClassifier(random_state=1)))
models.append(("Random forest", RandomForestClassifier(random_state=1)))
models.append(("GBM", GradientBoostingClassifier(random_state=1)))
models.append(("Adaboost", AdaBoostClassifier(random_state=1)))
models.append(("Xgboost", XGBClassifier(random_state=1, eval_metric="logloss")))
models.append(("dtree", DecisionTreeClassifier(random_state=1)))

results1 = [] # Empty list to store all model's CV scores
names = [] # Empty list to store name of the models

# Loop through all models to get the mean cross validated score
print("\n" "Cross-Validation performance on training dataset:" "\n")

for name, model in models:
    kfold = StratifiedKFold(
        n_splits=5, shuffle=True, random_state=1
    ) ## Complete the code to set the number of splits
    cv_result = cross_val_score(
        estimator=model, X=X_train_un, y=y_train_un, scoring = scorer, cv=kfold, n_jobs=1
    )
    results1.append(cv_result)
    names.append(name)
    print("{}: {}".format(name, cv_result.mean()))

print("\n" "Validation Performance:" "\n")

for name, model in models:
    model.fit(X_train_un, y_train_un) ## Complete the code to fit the model on unde

    # scores = accuracy_score(y_val, model.predict(X_val)) ## uncomment this line i
    # scores = recall_score(y_val, model.predict(X_val)) ## uncomment this line in
    # scores = precision_score(y_val, model.predict(X_val)) ## uncomment this line
    scores = f1_score(y_val, model.predict(X_val)) ## uncomment this line in case t

    print("{}: {}".format(name, scores))

```

Cross-Validation performance on training dataset:

Bagging: 0.6411413525524321
Random forest: 0.6875011408129813
GBM: 0.7131358906535971
Adaboost: 0.7002423692713601
Xgboost: 0.6944693136408734
dtree: 0.617022679979161

Validation Performance:

Bagging: 0.6916956737941323
Random forest: 0.734144015259895
GBM: 0.7608695652173914
Adaboost: 0.7559219693450998
Xgboost: 0.7423652871123688
dtree: 0.6839080459770115

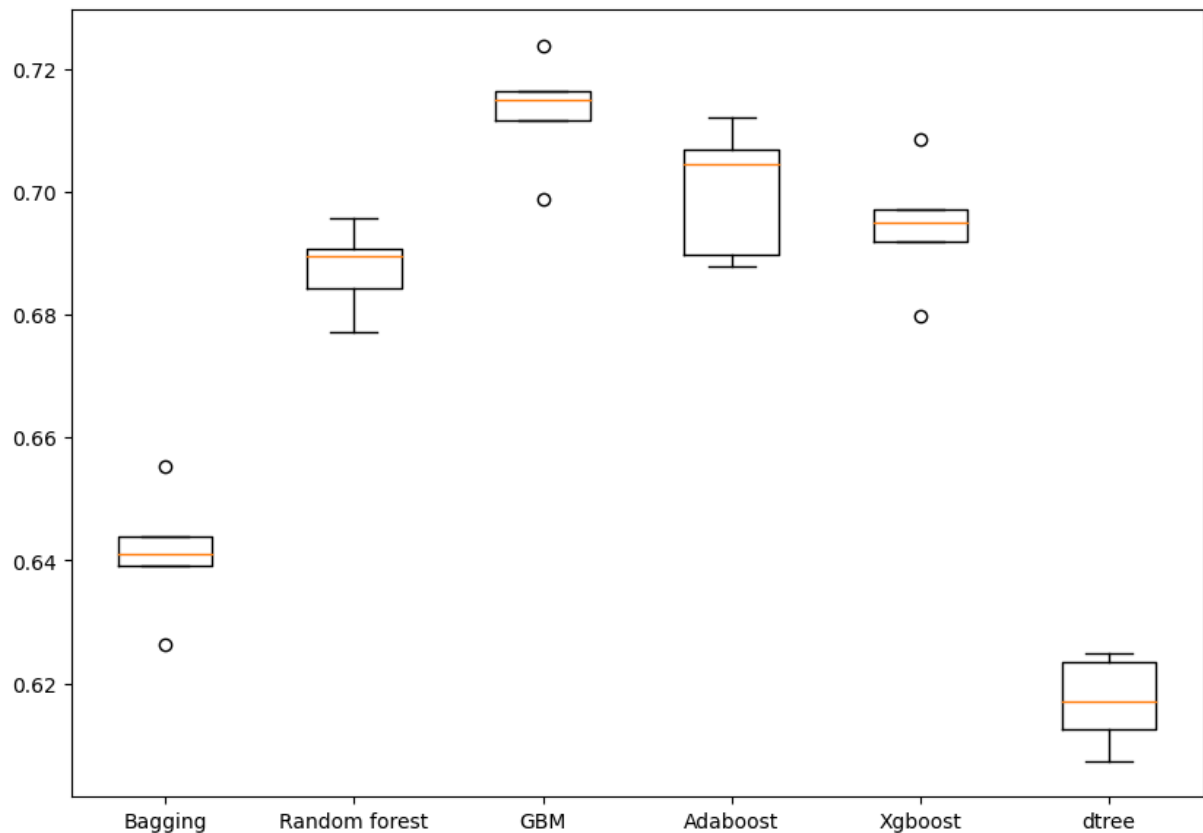
```
In [206... # Plotting boxplots for CV scores of all models defined above
fig = plt.figure(figsize=(10, 7))

fig.suptitle("Algorithm Comparison")
ax = fig.add_subplot(111)

plt.boxplot(results1)
ax.set_xticklabels(names)

plt.show()
```

Algorithm Comparison



Hyperparameter Tuning

Tuning AdaBoost using Oversampled data

Best practices for hyperparameter tuning in AdaBoost:

`n_estimators` :

- Start with a specific number (50 is used in general) and increase in steps: 50, 75, 100, 125
- Use fewer estimators (e.g., 50 to 100) if using complex base learners (like deeper decision trees)
- Use more estimators (e.g., 100 to 150) when learning rate is low (e.g., 0.1 or lower)
- Avoid very high values unless performance keeps improving on validation

`learning_rate` :

- Common values to try: 1.0, 0.5, 0.1, 0.01
- Use 1.0 for faster training, suitable for fewer estimators

- Use 0.1 or 0.01 when using more estimators to improve generalization
- Avoid very small values (< 0.01) unless you plan to use many estimators (e.g., > 500) and have sufficient data

In [208...

```
%%time

# defining model
model = AdaBoostClassifier(random_state=1)

# Parameter grid to pass in RandomSearchCV
param_grid = {
    "n_estimators": [50, 75, 100, 125, 150], ## Complete the code to set the number
    "learning_rate": [0.01, 0.1, 0.5, 1.0], ## Complete the code to set the Learnin
    "estimator": [DecisionTreeClassifier(max_depth=1, random_state=1), DecisionTree
]
}

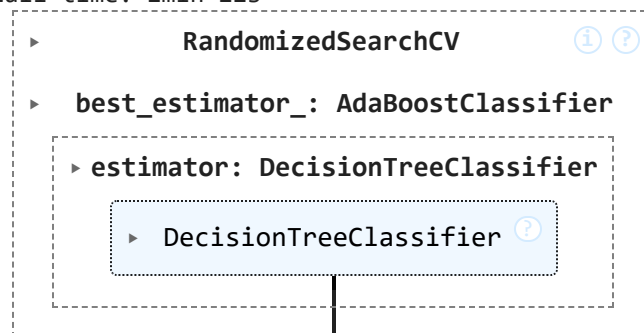
#Calling RandomizedSearchCV
randomized_cv = RandomizedSearchCV(
    estimator=model,
    param_distributions=param_grid,
    n_iter=50,
    n_jobs=-2,
    scoring=scorer,
    cv=5, ## Complete the code to set the cv parameter
    random_state=1
)

#Fitting parameters in RandomizedSearchCV
randomized_cv.fit(X_train_over, y_train_over) ## Complete the code to fit the model
```

CPU times: total: 8.53 s

Wall time: 2min 22s

Out[208...



In [209...

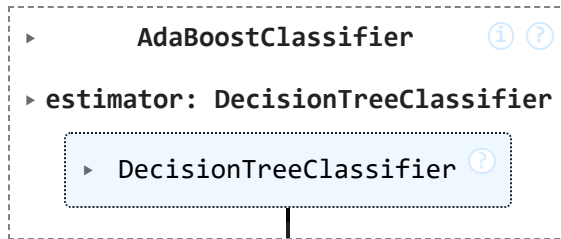
```
print("Best parameters are {} with CV score={}:".format(randomized_cv.best_params_
```

Best parameters are {'n_estimators': 150, 'learning_rate': 0.1, 'estimator': DecisionTreeClassifier(max_depth=3, random_state=1)} with CV score=0.7957153990629285:

In [210...

```
tuned_ada = randomized_cv.best_estimator_
tuned_ada
```

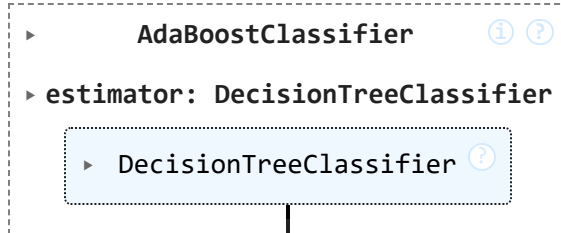
Out[210...



In [211...

```
tuned_ada.fit(X_train_over, y_train_over)
```

Out[211...



In [212...

```
tuned_ada_train_perf = model_performance_classification_sklearn(tuned_ada, X_train_
tuned_ada_train_perf
```

Out[212...

	Accuracy	Recall	Precision	F1
0	0.801771	0.821036	0.790575	0.805518

In [213...

```
## Complete the code to check the model performance for validation data.
tuned_ada_val_perf = model_performance_classification_sklearn(tuned_ada, X_val, y_v
tuned_ada_val_perf
```

Out[213...

	Accuracy	Recall	Precision	F1
0	0.72685	0.81084	0.786695	0.798585

Tuning Random forest using Undersampled data

Best practices for hyperparameter tuning in Random Forest:

`n_estimators` :

- Start with a specific number (50 is used in general) and increase in steps: 50, 75, 100, 125
- Higher values generally improve performance but increase training time
- Use 100-150 for large datasets or when variance is high

`min_samples_leaf` :

- Try values like: 1, 2, 4, 5, 10
- Higher values reduce model complexity and help prevent overfitting
- Use 1–2 for low-bias models, higher (like 5 or 10) for more regularized models
- Works well in noisy datasets to smooth predictions

`max_features` :

- Try values: `"sqrt"` (default for classification), `"log2"`, `None`, or float values (e.g., `0.3`, `0.5`)
- `"sqrt"` balances between diversity and performance for classification tasks
- Lower values (e.g., `0.3`) increase tree diversity, reducing overfitting
- Higher values (closer to `1.0`) may capture more interactions but risk overfitting

`max_samples` (for bootstrap sampling):

- Try float values between `0.5` to `1.0` or fixed integers
- Use `0.6-0.9` to introduce randomness and reduce overfitting
- Smaller values increase diversity between trees, improving generalization

In [215...

```
%%time

# defining model
model = RandomForestClassifier(random_state=1)

# Parameter grid to pass in RandomSearchCV
param_grid = {
    "n_estimators": [50, 75, 100, 125, 150], ## Complete the code to set the number
    "min_samples_leaf": [1, 2, 4, 5, 10], ## Complete the code to set the minimum n
    "max_features": ["sqrt", "log2", 0.3, 0.5, None], ## Complete the code to set t
    "max_samples": [0.6, 0.7, 0.8, 0.9, 1.0], ## Complete the code to set the maxim
}

#Calling RandomizedSearchCV
randomized_cv = RandomizedSearchCV(
    estimator=model,
    param_distributions=param_grid,
    n_iter=50,
    n_jobs=-2,
    scoring=scorer,
    cv=5, ## Complete the code to set the cv parameter
    random_state=1
)

#Fitting parameters in RandomizedSearchCV
randomized_cv.fit(X_train_un, y_train_un) ## Complete the code to fit the model on
```

CPU times: total: 2.22 s

Wall time: 1min 20s

Out[215...

```
RandomizedSearchCV
  best_estimator_: RandomForestClassifier
    RandomForestClassifier
```

```
In [216...] print("Best parameters are {} with CV score={}: ".format(randomized_cv.best_params_
```

Best parameters are {'n_estimators': 100, 'min_samples_leaf': 10, 'max_samples': 0.7, 'max_features': 'log2'} with CV score=0.7223779733598498:

```
In [217...] tuned_rf = randomized_cv.best_estimator_
tuned_rf
```

```
Out[217...] RandomForestClassifier
RandomForestClassifier(max_features='log2', max_samples=0.7,
                        min_samples_leaf=10, random_state=1)
```

```
In [218...] tuned_rf.fit(X_train_over, y_train_over)
```

```
Out[218...] RandomForestClassifier
RandomForestClassifier(max_features='log2', max_samples=0.7,
                        min_samples_leaf=10, random_state=1)
```

```
In [219...] tuned_rf_train_perf = model_performance_classification_sklearn(
    tuned_rf, X_train_un, y_train_un
)
tuned_rf_train_perf
```

```
Out[219...]
Accuracy  Recall  Precision  F1
0  0.727587  0.8364  0.686911  0.754321
```

```
In [220...] ## Complete the code to print the model performance on the validation data.
tuned_rf_val_perf = model_performance_classification_sklearn(tuned_rf, X_val, y_val)
tuned_rf_val_perf
```

```
Out[220...]
Accuracy  Recall  Precision  F1
0  0.730775  0.814758  0.788997  0.801671
```

Tuning with Gradient boosting with Oversampled data

Best practices for hyperparameter tuning in Gradient Boosting:

n_estimators :

- Start with 100 (default) and increase: 100, 125, 150, 175
- Typically, higher values lead to better performance, but they also increase training time
- Use 150-250 for larger datasets or complex problems
- Monitor validation performance to avoid overfitting, as too many estimators can degrade generalization

learning_rate :

- Common values to try: 0.1, 0.05, 0.01, 0.005
- Use lower values (e.g., 0.01 or 0.005) if you are using many estimators (e.g., > 200)
- Higher learning rates (e.g., 0.1) can be used with fewer estimators for faster convergence
- Always balance the learning rate with **n_estimators** to prevent overfitting or underfitting

subsample :

- Common values: 0.7, 0.8, 0.9, 1.0
- Use a value between **0.7** and **0.9** for improved generalization by introducing randomness
- **1.0** uses the full dataset for each boosting round, potentially leading to overfitting
- Reducing **subsample** can help reduce overfitting, especially in smaller datasets

max_features :

- Common values: **"sqrt"**, **"log2"**, or float (e.g., **0.3**, **0.5**)
- **"sqrt"** (default) works well for classification tasks
- Lower values (e.g., **0.3**) help reduce overfitting by limiting the number of features considered at each split

In [221...

```

%%time

# defining model
model = GradientBoostingClassifier(random_state=1)

## Complete the code to define the hyper parameters.
param_grid={
    "n_estimators": [100, 125, 150, 175, 200], ## Complete the code to set the numb
    "learning_rate": [0.005, 0.01, 0.05, 0.1], ## Complete the code to set the Lear
    "subsample": [0.7, 0.8, 0.9, 1.0], ## Complete the code to set the value for sub
    "max_features": ["sqrt", "log2", 0.3, 0.5, None] ## Complete the code to set the
}

#Calling RandomizedSearchCV
randomized_cv = RandomizedSearchCV(
    estimator=model,
    param_distributions=param_grid,
    n_iter=50,
    n_jobs=-2,
    scoring=scorer,
    cv=5, ## Complete the code to set the cv parameter
    random_state=1
)

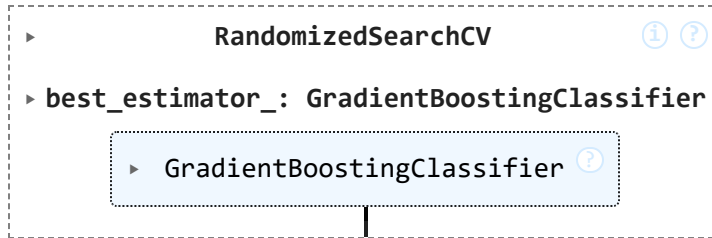
#Fitting parameters in RandomizedSearchCV
randomized_cv.fit(X_train_over, y_train_over)

```

CPU times: total: 7.5 s

Wall time: 1min 46s

Out[221...



In [222...

```
tuned_gbm = randomized_cv.best_estimator_
tuned_gbm
```

Out[222...

```

GradientBoostingClassifier
GradientBoostingClassifier(n_estimators=200, random_state=1, subsample=0.8)

```

In [223...

```

tuned_gbm_train_perf = model_performance_classification_sklern(
    tuned_gbm, X_train_over, y_train_over
)
tuned_gbm_train_perf

```

Out[223...

	Accuracy	Recall	Precision	F1
0	0.805633	0.828003	0.792544	0.809885

In [225...

```

## Complete the code to print the model performance on the validation data.
tuned_gbm_val_perf = model_performance_classification_sklern(tuned_gbm, X_val, y_v
tuned_gbm_val_perf

```

Out[225...

	Accuracy	Recall	Precision	F1
0	0.728885	0.816935	0.78564	0.800982

[OPTIONAL] Tuning XGBoost using Oversampled data

Best practices for hyperparameter tuning in XGBoost:

`n_estimators` :

- Start with 50 and increase in steps: 50,75,100,125.
- Use more estimators (e.g., 150-250) when using lower learning rates
- Monitor validation performance
- High values improve learning but increase training time

`subsample` :

- Common values: 0.5, 0.7, 0.8, 1.0

- Use `0.7-0.9` to introduce randomness and reduce overfitting
- `1.0` uses the full dataset in each boosting round; may overfit on small datasets
- Values < 0.5 are rarely useful unless dataset is very large

`gamma` :

- Try values: 0 (default), 1, 3, 5, 8
- Controls minimum loss reduction needed for a split
- Higher values make the algorithm more conservative (i.e., fewer splits)
- Use values > 0 to regularize and reduce overfitting, especially on noisy data

`colsample_bytree` :

- Try values: 0.3, 0.5, 0.7, 1.0
- Fraction of features sampled per tree
- Lower values (e.g., 0.3 or 0.5) increase randomness and improve generalization
- Use `1.0` when you want all features considered for every tree

`colsample_bylevel` :

- Try values: 0.3, 0.5, 0.7, 1.0
- Fraction of features sampled at each tree level (i.e., per split depth)
- Lower values help in regularization and reducing overfitting
- Often used in combination with `colsample_bytree` for fine control over feature sampling

In [226...

```
%%time

# defining model
model = XGBClassifier(random_state=1,eval_metric='logloss')

## Complete the code to define the hyperparameters
param_grid={
    'n_estimators':[50, 75, 100, 125, 150], ## Complete the code to set the number
    'subsample':[0.5, 0.7, 0.8, 1.0], ## Complete the code to set the subsample.
    'gamma':[0, 1, 3, 5, 8], ## Complete the code to set the gamma.
    'colsample_bytree':[0.3, 0.5, 0.7, 1.0], ## Complete the code to set the value
    'colsample_bylevel':[0.3, 0.5, 0.7, 1.0], ## Complete the code to set the value
}

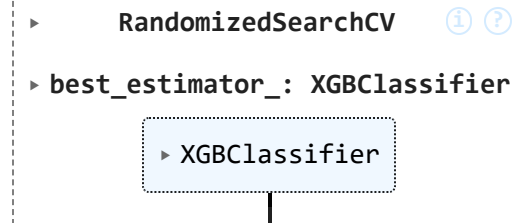
#Calling RandomizedSearchCV
randomized_cv = RandomizedSearchCV(
    estimator=model,
    param_distributions=param_grid,
    n_iter=50,
    n_jobs=-2,
    scoring=scorer,
    cv=5, ## Complete the code to set the cv parameter
    random_state=1
)
```

```
#Fitting parameters in RandomizedSearchCV
randomized_cv.fit(X_train_over,y_train_over)## Complete the code to fit the model o
```

CPU times: total: 3.94 s

Wall time: 18.5 s

Out[226...



In [227...

```
print("Best parameters are {} with CV score={}" .format(randomized_cv.best_params_
```

Best parameters are {'subsample': 0.8, 'n_estimators': 150, 'gamma': 5, 'colsample_bytree': 1.0, 'colsample_bylevel': 1.0} with CV score=0.8028357969498081:

In [228...

```
tuned_xgb = randomized_cv.best_estimator_
tuned_xgb
```

Out[228...

XGBClassifier

XGBClassifier(base_score=None, booster=None, callbacks=None, colsample_bylevel=1.0, colsample_bynode=None, colsample_bytree=1.0, device=None, early_stopping_rounds=None, enable_categorical=False, eval_metric='logloss', feature_types=None, gamma=5, grow_policy=None, importance_type=None, interaction_constraints=None, learning_rate=None, max_bin=None, max_cat_threshold=Non

In [229...

```
tuned_xgb_train_perf = model_performance_classification_sklern(
    tuned_xgb, X_train_over, y_train_over
)
tuned_xgb_train_perf
```

Out[229...

	Accuracy	Recall	Precision	F1
0	0.806976	0.84093	0.787455	0.813314

In [231...

```
## Complete the code to print the model performance on the validation data.
tuned_xgb_val_perf = model_performance_classification_sklern(tuned_xgb, X_val, y_v
tuned_xgb_val_perf
```

Out[231...

	Accuracy	Recall	Precision	F1
0	0.731792	0.827384	0.783227	0.8047

We have now tuned all the models, let's compare the performance of all tuned models and see which one is the best.

Model Performance Summary and Final Model Selection

```
In [232... # training performance comparison

models_train_comp_df = pd.concat(
    [
        tuned_gbm_train_perf.T,
        # tuned_xgb_train_perf.T, ## uncomment this line if XGBoost model was tuned
        tuned_ada_train_perf.T,
        tuned_rf_train_perf.T,
    ],
    axis=1,
)
models_train_comp_df.columns = [
    "Gradient Boosting tuned with oversampled data",
    # "XGBoost tuned with oversampled data", ## uncomment this line if XGBoost mode
    "AdaBoost tuned with oversampled data",
    "Random forest tuned with undersampled data",
]
print("Training performance comparison:")
models_train_comp_df
```

Training performance comparison:

	Gradient Boosting tuned with oversampled data	AdaBoost tuned with oversampled data	Random forest tuned with undersampled data
Accuracy	0.805633	0.801771	0.727587
Recall	0.828003	0.821036	0.836400
Precision	0.792544	0.790575	0.686911
F1	0.809885	0.805518	0.754321

```
In [233... # validation performance comparison

models_val_comp_df = pd.concat(
    [
        tuned_gbm_val_perf.T,
        # tuned_xgb_val_perf.T, ## uncomment this line if XGBoost model was tuned
        tuned_ada_val_perf.T,
        tuned_rf_val_perf.T,
    ],
    axis=1,
)
models_val_comp_df.columns = [
    "Gradient Boosting tuned with oversampled data",
    # "XGBoost tuned with oversampled data", ## uncomment this line if XGBoost mode
    "AdaBoost tuned with oversampled data",
```

```

    "Random forest tuned with undersampled data",
]
print("Validation performance comparison:")
models_val_comp_df

```

Validation performance comparison:

Out[233...

	Gradient Boosting tuned with oversampled data	AdaBoost tuned with oversampled data	Random forest tuned with undersampled data
Accuracy	0.728885	0.726850	0.730775
Recall	0.816935	0.810840	0.814758
Precision	0.785640	0.786695	0.788997
F1	0.800982	0.798585	0.801671

In [234...

```

# selecting the final model by uncommenting the line corresponding to the final mod

final_model = tuned_gbm ## uncomment this line if the final model chosen is Gradient
# final_model = tuned_xgb ## uncomment this line if the final model chosen is XGBoost
# final_model = tuned_ada ## uncomment this line if the final model chosen is AdaBoost
# final_model = tuned_rf ## uncomment this line if the final model chosen is Random

```

In [235...

```

test = model_performance_classification_sklern(final_model, X_test, y_test)
test

```

Out[235...

	Accuracy	Recall	Precision	F1
0	0.756863	0.845401	0.801484	0.822857

In [251...

```

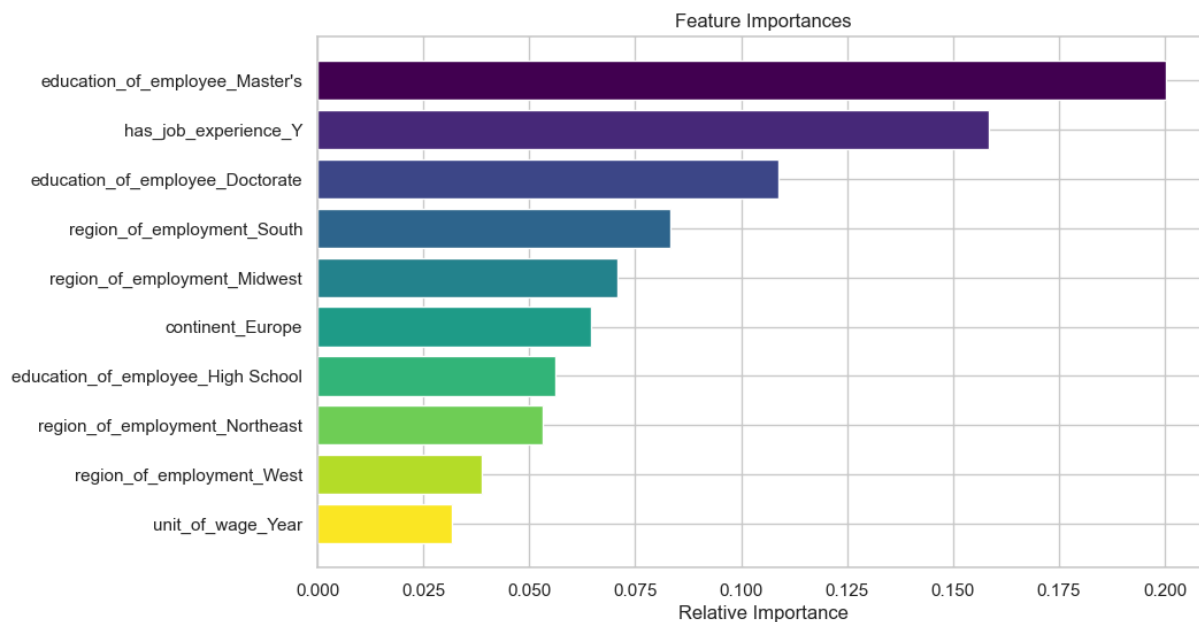
feature_names = X_train.columns
importances = final_model.feature_importances_
indices = np.argsort(importances)[-10:] ## Complete the code to get the indices of

# Spicing up the plot with different colors and size
# Pick a palette (colormap)
cmap = plt.cm.viridis

# Create one color per bar (from High -> Low)
colors = cmap(np.linspace(1, 0, len(indices)))

plt.figure(figsize=(10, 6))
plt.title("Feature Importances")
plt.barh(range(len(indices)), importances[indices], color=colors, align="center")
plt.yticks(range(len(indices)), [feature_names[i] for i in indices])
plt.xlabel("Relative Importance")
plt.show()

```



Actionable Insights and Recommendations

- This section can be attempted directly in the presentation.

Power Ahead
